

AI Research Preference Models

Thomas Simon Foster^{1,2,†}, Bassel Al Omari^{1,†}, Tingchen Fu^{1,2,†}, Thomas Mann^{1,*}, Carl Domond^{1,*}, Lucia Cipolina-Kun¹, Bhavul Gauri¹, Muna Aghamelu¹, Alexander D. Goldie^{1,2}, Eryk Helenowski¹, Jean-Christophe Gagnon-Audet¹, Alberto Pepe¹, Saba Nazir¹, Daniel Izcovich¹, Noam Levi¹, Rishi Hazra¹, Karen Hambardzumyan^{1,3}, Nicolas Baldwin¹, Xian Li¹, Martin Josifoski¹, Paris Giampouras¹, Masoud Jalili Sabet¹, Anya Sims¹, Hela Momand¹, Tatiana Shavrina¹, Despoina Magka¹, Jason Weston¹, Yulin Wang², Anirudh Goyal¹, João Henriques², Yoram Bachrach¹, Emily McMilin^{1,‡}, Jakob Nicolaus Foerster^{1,2,‡}

¹FAIR at Meta, ²University of Oxford, ³University College London

[†]Lead Authors, ^{*}Core Contribution, [‡]Equal Supervision

AI research agents (AIRA) can now carry machine learning experiments from proposal through implementation and evaluation. Yet progress on frontier tasks is throttled by the cost of evaluations that can consume days of GPU time. When an agent can propose far more candidates than it can afford to run, progress depends on its *research preference*: how it allocates a fixed execution budget across many candidates. We introduce AI Research Preference Models (RPMs) that predict which candidate solution is most promising, without paying the cost of running them all. We build RPMs from frozen pretrained language models in two variants: an inference-only model that reasons over candidate plans, code, and previously executed solutions, and an agentic model that additionally runs small-scale pilot experiments. Integrated into the AIRA-dojo research agent and evaluated on the machine learning research benchmark AIRS-BENCH, the two variants increase the average normalized score from 0.684 to 0.711 and 0.729, respectively. Both reach the unguided agent’s 24-hour performance in roughly 15 hours, using less than two-thirds of its execution budget, and together yield new state-of-the-art results on two AIRS-BENCH tasks.

Correspondence: Bassel Al Omari at balomari@meta.com



1 Introduction

Language model agents have advanced rapidly in domains such as mathematics, coding, and computer use, where candidate actions can be evaluated accurately and efficiently: a mathematical answer can be checked against a reference, a program run against a test suite, and a computer-use task verified against its target state. These evaluation functions provide the reward signal that lets agents iterate and improve, driving rapid progress on benchmarks such as SWE-bench (Jimenez et al., 2024) and Terminal Bench (Merrill et al., 2026).

Progress has been slower for AI research agents (AIRAs) that autonomously propose, implement, and evaluate their own experiments, despite recent efforts such as AIDE (Jiang et al., 2025), AIRA-dojo (Toledo et al., 2025), AIRA₂ (Hambardzumyan et al., 2026) and MARS (Chen et al., 2026). Frontier machine learning research lacks cheap feedback: proposing or modifying candidate code can be quick, but executing it to train a model and measuring its performance can consume hours to days or decades of GPU time. Considering an agent can propose far more candidates than it can afford to run, the primary lever on research progress becomes *research preference*: deciding which research directions are promising enough to allocate compute budget to, and which directions to drop.

We address this allocation problem directly by introducing a dedicated **AI Research Preference Model (RPM)** into an AIRA. The RPM receives the full context of previously executed solutions, and uses it to decide which of multiple newly generated candidate solutions will be most valuable to execute next.

We summarize our contributions below:

- We introduce AI Research Preference Models, a framework for efficiently allocating compute to the most promising candidate solutions in AI research agents.

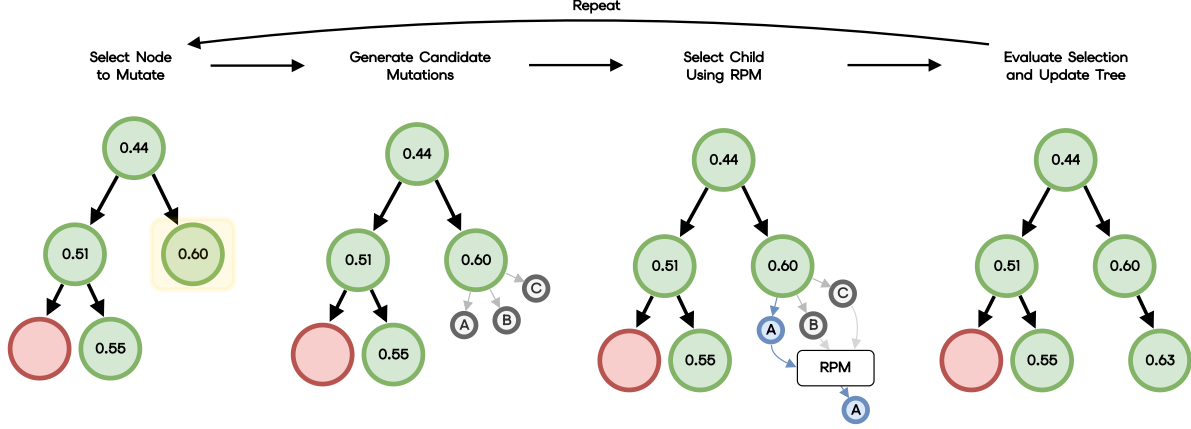


Figure 1: Overview of an AI research agent with Research Preference Model (RPM) augmented child creation. Each node represents a solution within the search tree, labeled with its evaluation score. The agent (1) selects a promising parent node from the active tree, (2) generates a batch of candidate child solutions, (3) utilizes the RPM to select the most promising candidate, using the full context of previously executed solutions and (4) executes and scores only the selected candidate to expand the tree, to reduce the compute overhead of running unpromising candidates (Section 3.3).

- We develop *Inference-only RPMs*, frozen-weight LLMs that reason over candidate plans, code and previous solutions, and demonstrate that it raises performance of the AIRA-dojo agent on AIRS-BENCH from 0.684 to 0.711.
- We further propose *Agentic RPMs*, an extension of Inference-only RPMs with the ability to run small-scale pilot experiments. Integrated within AIRA-dojo, Agentic RPMs further raise performance on AIRS-BENCH to 0.729, approaching the validation oracle ceiling of 0.748.
- On AIRS-BENCH, agents equipped with our best RPMs yield new state-of-the-art results on two tasks and match the unguided agent’s 24-hour performance in roughly 15 hours, using less than two-thirds of its execution budget (Fig. 2).

2 Background

In line with the broader agent research literature, we view an agent as a computer system that is situated in some environment and is able to act autonomously in this environment in order to achieve its design objectives (Wooldridge and Jennings, 1995). In our setting, an AI research agent acts by generating and executing code. The objective is to produce an artifact (such as model weights, an optimised code snippet, or an answer to a question) that, when evaluated by some task-specific reward function, achieves a high score.

2.1 AI Research Agent Benchmarks

Recent benchmarks evaluate large language model agents across complex, long-horizon workflows. For software engineering, popular suites like SWE-bench (Jimenez et al., 2024) assess an agent’s ability to resolve real GitHub issues and modify multi-file codebases. Within the data science domain, benchmarks like MLE-bench (Chan et al., 2025) evaluate agent capabilities through structured machine learning engineering competitions.

We base our experiments on AIRS-BENCH (Lupidi et al., 2026), a comprehensive suite of 20 machine learning tasks sourced from state-of-the-art papers. These tasks span diverse domains, including language modeling, mathematics, bioinformatics, and time series forecasting. AIRS-BENCH assesses agentic capabilities over the full research lifecycle (including idea generation, experiment analysis, and iterative refinement) without providing baseline code. Each task is rigorously specified by a problem, a dataset, a target metric, and a published state-of-the-art value.

Here the agents are placed in an environment with a training dataset, a set of test inputs, and 24 hours of access to an H200 GPU. The AIRA’s goal is to write code that trains a model and produces a `submission.csv` with predictions on the test inputs. The agent may choose to run code that reports a validation score (e.g., from using cross-validation) that it can use for guiding search. The true test score (produced by comparing the `submission.csv` to the ground truth labels) is hidden from the agent.

To aggregate performance across heterogeneous task metrics, AIRS-BENCH defines a Normalized Score for agent a on task t :

$$\text{NS}_t^a = \frac{\phi_t(s_t^a) - \phi_t(s_t^{\min})}{\phi_t(s_t^{\text{sota}}) - \phi_t(s_t^{\min})}, \quad (1)$$

where $s_t^{\min}$ is the worst score observed across all agents, s_t^{sota} is the most recent public SOTA score as of the benchmark’s publication, and ϕ_t is a non-linear log transform, defined as $\phi_t(s) = -\log_{10}(|s - s_t^{\text{opt}}|)$ to properly weight exponential progress near optimal bounds (s_t^{opt}). Under this metric, $\text{NS} = 0$ corresponds to the minimum baseline and $\text{NS} = 1$ to the public SOTA, while invalid or failed submissions receive a score of 0. The Average Normalized Score reported on this benchmark averages the Normalized Score uniformly across all tasks and seeds.

2.2 AI Research Agent Scaffolds

To systematically analyze and improve AI research agents, we decompose an AIRA into an LLM backbone and an algorithmic *scaffold*. While the backbone provides core reasoning capabilities, the scaffold defines the decision logic and search strategy (ranging from simple linear loops to complex tree search) that govern how candidate solutions are generated, evaluated, and iteratively refined. A single scaffold like AIRA-dojo (Toledo et al., 2025) or Claude Code (Anthropic, 2025) can be instantiated with different backbones, such as Claude Opus (Anthropic, 2026) or GPT-5 (Singh et al., 2025).

These candidate solutions can be viewed as nodes within an evolving solution graph, where each node stores concrete artifacts like code scripts, execution logs, and metric scores. The agent explores this graph by selecting a parent node and mutating its contents to produce a child node. Inspired by this evolutionary computation framework, we characterize the core design space of search scaffolds along three primary axes:

- **Parent Selection:** The strategy used to identify which historical solutions, trajectories, or ideas are most promising to build upon next.
- **Child Creation:** The process of taking the selected parent solutions and prompting the LLM to generate a new candidate child solution. The localized prompts used to drive *child creation* are referred to as *operators*.
- **Final Solution Selection:** The criteria used to evaluate the accumulated bank of candidate solutions and determine which single solution to return as the final output.

Existing AIRA architectures implement these mechanisms in fundamentally different ways. For example, MLGYM (Nathani et al., 2025) operates as a linear search scaffold that heavily simplifies parent selection by always choosing the most recent node as the parent, relying on a single mutation operator to iteratively refine it. In contrast, we build upon AIRA-dojo, an evolutionary tree-search framework that uses greedy parent selection to always mutate the node with the highest current validation score. To orchestrate child creation, AIRA-dojo employs a specialized suite of mutation operators tailored to distinct engineering phases, specifically DRAFT, IMPROVE, and DEBUG. Finally, for final solution selection, AIRA-dojo submits the node that achieved the highest validation score across the entire search tree.

Ultimately, this loop produces an expanding bank of candidate solutions, each paired with its evaluation score. It is over this growing set that a preference model can intervene, deciding which candidates are worth the expense of execution. It also serves as a valuable dataset to train and evaluate an RPM.

3 AI Research Preference Models

Frontier machine learning research lacks low-cost feedback: while proposing candidate solutions is fast, executing them to train a model can consume hours or days of GPU compute. Faced with this bottleneck, research progress heavily relies on predicting the value of pursuing candidate research directions.

To address this challenge, we introduce AI Research Preference Models (RPMs) to guide experimental allocation within research agents. Through initial experimentation, we observed that language models perform unreliably when forecasting absolute metrics or execution outcomes. Consequently, an RPM reformulates experimental allocation as a preference ranking problem: ranking candidate solutions to select the most promising paths before dedicating compute to pursuing them.

We explore RPMs leveraging varying ranges of test-time compute:

- **Inference-only RPMs** (Section 3.1): Rank candidate solutions using lightweight reasoning over search history and code diffs.
- **Agentic RPMs** (Section 3.2): Allocate additional compute to run rapid sandbox pilot experiments prior to ranking.

Designed as a scaffold-agnostic component, RPMs can interface with a wide variety of AIRA architectures. In this work, we investigate integrating RPMs into the AIRA-dojo evolutionary tree-search scaffold (Section 3.3). We target the child-creation phase, where creating a single child mutation is replaced with generating N candidate modifications in parallel and using an RPM-guided tournament to select the most promising solution before committing GPU compute.

3.1 Inference-only RPMs

The “LLM-as-a-Judge” paradigm (Zheng et al., 2023) demonstrates that LLMs can rank technical solutions with reasonable fidelity using internal intuition and reasoning. Motivated by this approach, we experiment with purely querying pretrained LLMs as an inexpensive preference model to select between research ideas. To understand how visibility into the AIRA’s search space affects the RPM’s selection quality, we experiment with varying the count of previously explored solutions visible to the RPM and the count of suggestions it selects between.

To develop the chosen prompt even further, we leverage MIPROv2 from the DSPy framework (Opsahl-Ong et al., 2024), a widely adopted baseline for robustly optimizing prompt instructions. The prompt optimizer generated an instruction set that directs the RPM to conduct a more structured analysis of each solution and remain tolerant of minor, fixable issues. Further details on the prompt are provided in Appendix A.1.

3.2 Agentic RPMs

Motivation A core practice in software engineering, and machine learning research is rapid prototyping. To comprehensively understand and validate the potential or the feasibility of a novel idea, researchers tend to quickly run small-scale pilot experiments before launching a full-volume large-scale experiment. Inspired by how pilot experiments inform the possible outcome and assist decision making, we develop Agentic RPMs where an agent can use multiple predefined tools in a sandbox environment to conduct pilot experiments before making a decision.

Agentic Workflow Concretely, a pilot experiment is conducted via multi-turn interaction between the language model agent and the environment, interleaving chain-of-thought reasoning, tool calling, and receiving environment feedback (Yao et al., 2023). The sandbox environment for the Agentic RPM is an exact clone of the environment for AIRA, including the access to a single H200 GPU. Meanwhile, the agentic RPM has access to the training dataset and unlabeled test dataset, together with necessary pre-installed Python packages in this environment, similar to the AI research agent. To interact with this environment, we provide the agent with a set of tools: `python`, `bash`, and `submit_solution`. The `python` tool and `bash` tool allow executing any Python code or Bash code, respectively, and then return execution results. With the outcome of the pilot experiment, the agent can summarize and submit the experimental findings with the tool `submit_solution`.

Notably, the agent is only required to submit the summary and analysis of the pilot experiment, but not to make a final selection among candidate solutions. The `submit_solution` tool will return the remaining time budget. If the remaining time budget surpasses a specific threshold, the agent would be prompted to run further experiments to make the best use of the time budget and computation resources. Finally, the candidate solution, the task description, and all submitted pilot experiment findings are input to a language model to select the best solution. Full implementation details on agentic RPM are provided in Appendix B.

3.3 AIRA Integration

While an RPM can intercept multiple stages of an agent’s search scaffold, we focus our implementation on augmenting the *child-creation* phase, as illustrated in Figure 1. In an evolutionary AIRA scaffold, child creation fundamentally encompasses two sub-steps: child candidate creation and child candidate selection. By default, AIRA-dojo generates a single candidate solution during candidate creation, which is automatically selected to be executed, evaluated, and added to the search tree. We modify this pipeline by first expanding child candidate creation: a chosen parent node is mutated by applying operators N times independently in parallel to yield N unexecuted child candidate solutions. During child candidate selection, we then introduce the RPM which evaluates these N candidates alongside historical trajectory context, conducting pairwise comparisons in a tournament knockout structure to select the single candidate for full execution.

To ground each comparison in the search so far, we also provide the RPM selected context from the tree. Context nodes are pulled via a BFS traversal of the already-explored tree starting from the parent, collecting up to K non-buggy nodes from earlier in the search; each of these previously-evaluated solutions is presented alongside the validation score it obtained.

4 Experimental Setup

4.1 End-to-End Evaluation

We integrate RPMs into the child-creation phase of AIRA-dojo, and evaluate this augmented scaffold against the 20 publicly released AIRS-BENCH tasks that fall under the text and tabular modalities. Following the original AIRS-BENCH evaluation protocol, the AIRA-dojo evaluations are provided 24 hours of access to a single H200 per task and are repeated for 10 seeds. To avoid the generalization gap which undermined long-horizon search in AIRA-dojo, we integrate the Hidden Consistent Evaluation protocol from AIRA₂. Both the AIRA-dojo operator used to generate the child candidates and the RPMs share a Qwen3.6-27B (Qwen Team, 2026) backbone, standardizing our setup on a high-performing open-weights model for code reasoning. Maintaining an identical model across child creation and selection ensures that all observed improvements are driven by the framework rather than a stronger selection backbone.

To contextualize these results, we first compare against the default No-RPM baseline. In this setup, child selection defaults to uniform random selection among generated candidates, which corresponds to vanilla AIRA-dojo in expectation. We also compare against a Test Oracle and a Validation Oracle, that are constructed by executing all candidates at each step and choosing the highest scorer. Only the compute time of the selected candidate counts towards the 24-hour limit. While neither is viable online (the Test Oracle utilizes privileged test-set information, and the Validation Oracle requires a prohibitive compute overhead to execute every candidate), they serve as ceilings for greedy child selection.

4.2 Offline Evaluation

The full end-to-end evaluations detailed in Section 4.1 are computationally expensive, requiring 200 H200 GPUs for 24 hours. To enable quick iterations and guide development of our RPMs before running full end-to-end evaluations, we produce an offline evaluation dataset compiled from previous AIRA-dojo runs on a separate set of 40 unreleased AIRS-BENCH tasks from the image, video and audio modalities. The development and evaluation sets are split by modality to avoid task contamination. The previous AIRA-dojo runs followed the standard AIRS-BENCH evaluation protocol of dedicating 24 compute hours with a single H200 and evaluating 10 different seeds. The previous runs were conducted with gpt-oss-120b (OpenAI, 2025), GPT-4o (OpenAI, 2024) and CWM (Copet et al., 2025) as the LLM backbone. From these runs, we

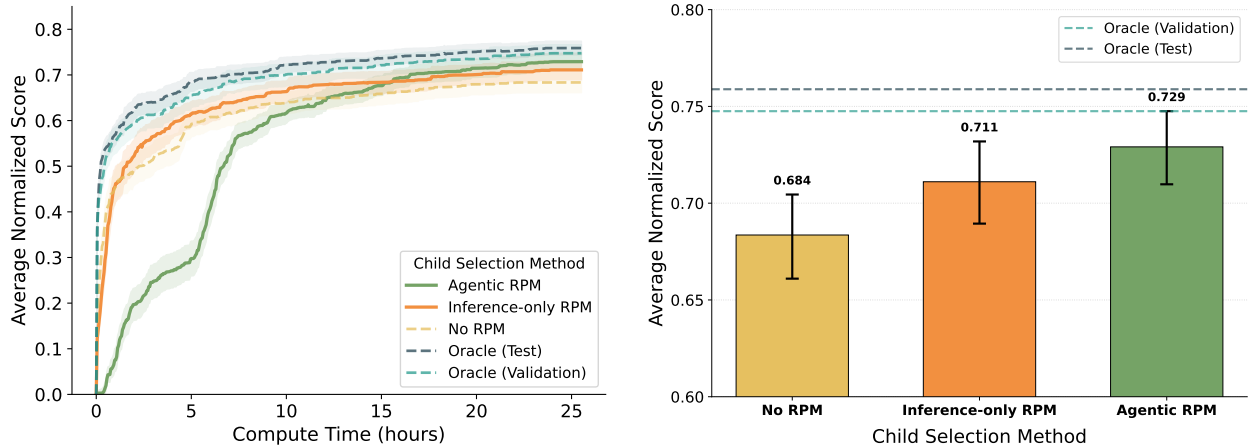


Figure 2: Evaluation of RPM-augmented AIRA-dojo. Average normalized scores over time (left) and final performance (right) for AIRA-dojo with RPM-augmented child selection on AIRS-BENCH, with error bars indicating the 95% confidence intervals. Dashed lines indicate the No-RPM baseline and immediate validation or test oracles. The Inference-only RPM (orange) yields steady gains over the baseline (yellow). While per-step proxy experiments slow its early trajectory, the Agentic RPM (green) leverages this compute overhead to surpass other methods.

extract 1,000 sibling node pairs, including their plans, code, and search tree history. In selecting these pairs, we discard near-ties with a normalized test-metric gap below 0.01, to prevent negligible, run-to-run metric noise from confounding the evaluation signal. The RPM is evaluated on its accuracy in selecting the node with the highest test score in its subtree. We choose this ground-truth label to ensure the RPMs look past immediate performance, explicitly rewarding candidates with strong long-term fixability and extensibility. We acknowledge that this label inherits a bias from the original greedy search policy, where nodes with stronger early scores are favored during search, expanding their subtrees and giving them greater opportunity to reach high scores. Random selection establishes a 50% baseline accuracy floor. In these offline evaluations, GPT-5 (Singh et al., 2025) is used as backbone for the RPMs.

5 Results

5.1 End-to-End Evaluations

We present the end-to-end performance of integrating our RPMs within AIRA-dojo, in Figure 2. In these evaluations, AIRA-dojo is configured to generate 15 child candidate suggestions at each operator step, leveraging our offline finding that expanding the candidate pool size systematically improves selection performance (Section 5.2.1). The Inference-only RPM evaluates these pairs using the “LLM-as-a-judge” reasoning prompt presented in Section 3.1, including the scores of historical nodes from the search tree. For the Agentic RPM detailed in Section 3.2, to balance overall compute budgets, we only deploy this selection mechanism for the DRAFT and IMPROVE operators, reverting to random selection during DEBUG steps.

As shown in Figure 2, our methods navigate the trade-off between decision quality and compute time in fundamentally different ways. Incurring no candidate-execution cost, the Inference-only RPM (orange) provides immediate and steady gains over the baseline with no RPM (yellow). Conversely, the Agentic RPM (green) exhibits a slow rise early in the run. Considering it runs small-scale proxy experiments in a sandbox at every step, its heavy per-step compute consumption slows early progress along the time axis. However, this rigorous per-step evaluation eventually triggers a sharp performance acceleration, ultimately matching or exceeding both the Inference-only RPM and the No-RPM baseline. Ultimately, both variants beat the unguided baseline’s final score of 0.684, with Inference-only reaching 0.711 and Agentic reaching 0.729, narrowing the gap toward the validation-oracle (0.748) and test-oracle (0.759) ceilings.

Significance Testing To evaluate statistical significance, we report the *probability of improvement*, defined as the likelihood that a randomly sampled run of one method outperforms another on a randomly selected task. We compute task-stratified bootstrap distributions using `rliable` (Agarwal et al., 2021), where a probability of 0.5 denotes no difference. The results show that the Inference-only RPM and the Agentic RPM achieve a statistically significant edge over the default AIRA-dojo baseline (No RPM), yielding average improvement probabilities of 0.5923 and 0.5913, respectively, with the 95% confidence intervals lower bounded at 0.5066 and 0.5018, strictly excluding the 0.5 mark of random chance.

State-of-the-Art Breakthroughs These guided search capabilities translate directly to new state-of-the-art (SOTA) milestones on established benchmarks, to the best of our knowledge. On WinoGrande (Sakaguchi et al., 2021), AIRA-dojo with the Agentic RPM achieves an accuracy of 94.1%, comfortably surpassing the previous agentic SOTA of 90.4% reported by Hambardzumyan et al. (2026). During this run, the agent fine-tunes a Qwen2.5-14B-Instruct model (Qwen Team, 2024) via LoRA on data with shuffled labels, then averages prediction logits across original and shuffled label orderings at inference to eliminate position bias. Similarly, on SVAMP (Patel et al., 2021), AIRA-dojo with the Inference-only RPM reaches 95.7% accuracy, eclipsing the prior human SOTA of 94.2% from Zhong et al. (2026). Here, the agent designs few-shot prompts that instruct the model to cleanly isolate relevant numerical data from distracting context, then generates ten independent reasoning paths by sampling Qwen2.5-7B-Instruct with increased temperature, and resolves the final prediction using a majority vote.

Research Efficiency Beyond absolute performance gains, both preference models significantly accelerate search velocity. While the unaugmented No-RPM baseline requires the full 24-hour allocation to reach its final score of 0.684, our RPM-guided approaches reach this identical performance threshold significantly faster. The Inference-only RPM matches this baseline score in 14.88 hours (a $1.61\times$ speedup), while the Agentic RPM achieves it in 15.50 hours (a $1.55\times$ speedup). This allows both methods to match standard performance while using approximately $1.5\times$ less compute budget.

Impact of Selection Quality To confirm how decision quality drives performance, we retrospectively analyze the selection advantage of each RPM throughout the runs. Selection advantage measures the average difference between the chosen candidate’s score and the overall batch mean. As expected, random selection yields a selection advantage of roughly 0.0 in expectation, whereas the Inference-only RPM achieves significantly higher selection quality, and the Agentic RPM yields the highest advantage on average. Crucially, we find a strong positive correlation between selection advantage and final normalized score (Pearson $r = 0.55$, Spearman $\rho = 0.56$), confirming that more accurate candidate selection translates to better end-to-end AIRA performance. Further details are provided in Appendix C.1.

5.2 Offline Evaluation and Further Analysis

To guide development of our RPMs before running full end-to-end evaluations, we leverage the offline framework from Section 4. Full 24-hour online runs are too computationally expensive for quick iteration, and thus this offline setup allows us to tune our models, analyze their core behaviors, and derive clear trends.

We observe that the best configuration of Inference-only RPMs is surpassed in predictive accuracy by our Agentic RPMs, with both exceeding the random baseline, matching the trend ultimately observed in our end-to-end evaluations.

5.2.1 Inference-only RPM

Inference Scaling We structure our offline analysis around three key dimensions that dictate the RPM’s visibility and analytical capacity: *context size* (the number of historical code solutions and validation scores provided from the tree), *suggestion count* (the number of candidate child solutions), and *reasoning budget* provided to the LLM.

Our offline tests reveal three clear scaling behaviors. First, *context scaling* demonstrates that supplying the RPM with a deeper history of historical search tree nodes and their validation scores consistently improves child-selection judgment. Second, *suggestion scaling* demonstrates that expanding the candidate

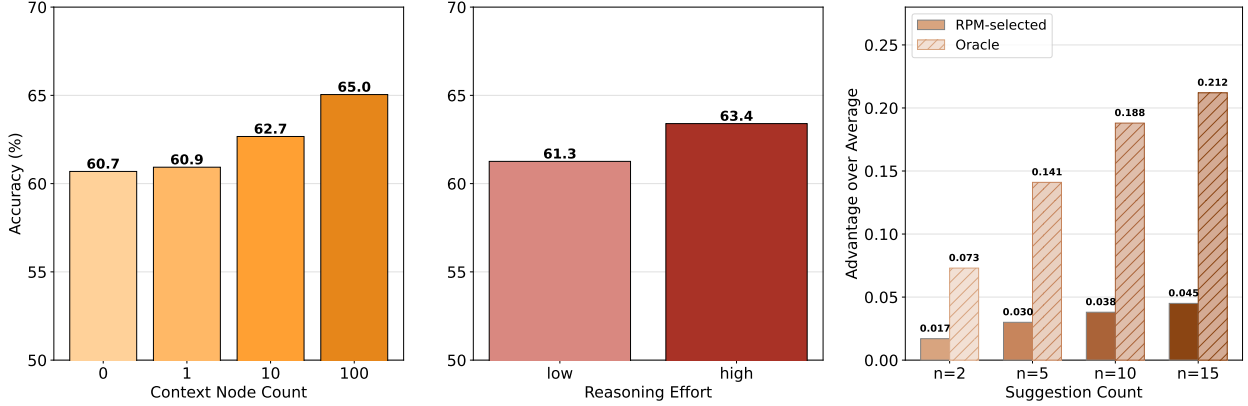


Figure 3 : Offline evaluation of Inference-only RPM scaling properties. Increasing search-tree context provided gives improved predictive accuracy (left). Allocating a higher reasoning budget yields better selection (middle). Expanding the candidate suggestion pool reliably improves both the oracle-selection limit and the RPM’s selection performance (right), measured by the average difference between the selected solution’s score and the average score of all candidates.

pool systematically increases the selection advantage, measured as the average difference between the selected candidate’s score and the overall batch mean. The oracle’s selection advantage rises significantly with pool size, a trend our RPM successfully captures to extract higher-quality solutions from larger candidate batches. Third, *reasoning scaling* reveals that increasing the reasoning budget allocated to the LLM judge steadily improves its selection accuracy.

These results motivated using a large suggestion count (15 suggestions) and to maximize the context nodes provided (by providing the maximum amount that can fit within the LLM’s context window), and high reasoning budget parameters for the RPM in our end-to-end evaluation.

Ensembling We evaluate three frontier models, GPT-5 (Singh et al., 2025), Claude Opus 4.8 (Anthropic, 2026) and Gemini 3.1 Pro (Google DeepMind, 2026), individually and via two aggregation techniques: a mechanical majority vote and an LLM-Arbiter ensemble that ingests the reasoning traces of all three models before making a final decision. Individual models’ performances range between 64.66% to 67.44% accuracy. Aggregating their diverse reasoning traces further mitigates errors where majority vote increases accuracy to 68.04%, while the LLM-Arbiter ensemble achieves the highest overall offline accuracy of 69.35%. A full breakdown of these configurations and their corresponding results is compiled in Appendix A.2.

Reasoning Analysis To get a better understanding of the RPM’s decision-making, we analyze its generated reasoning traces. We find that referencing prior evidence, correctness of implementation or the pretrained backbone in the justification yields higher selection accuracy compared to when these are omitted. We also find, that citing more unique values from historical context also yields improved selection accuracy. Further details on the reasoning analysis are provided in Appendix A.3.

5.2.2 Agentic RPM

Compute Scaling We analyze how the agentic RPM scales with execution time limits on our offline benchmark. Increasing the available compute budget consistently improves selection accuracy, scaling from 78.52% under a 5-minute constraint, to 82.78% at 30 minutes, and peaking at 84.02% with a 4-hour allocation. This scaling behavior demonstrates that allowing the agent more time to run, observe, and debug directly translates to higher-fidelity RPM estimations. Notably, the marginal returns obtained from increasing the time budget are relatively minimal considering the pilot-experiment agent shares the same time budget with the AI research agent and a 4-hour run of the Agentic RPM is prohibitively expensive. Therefore, we keep the time budget for end-to-end evaluation at 5 minutes.

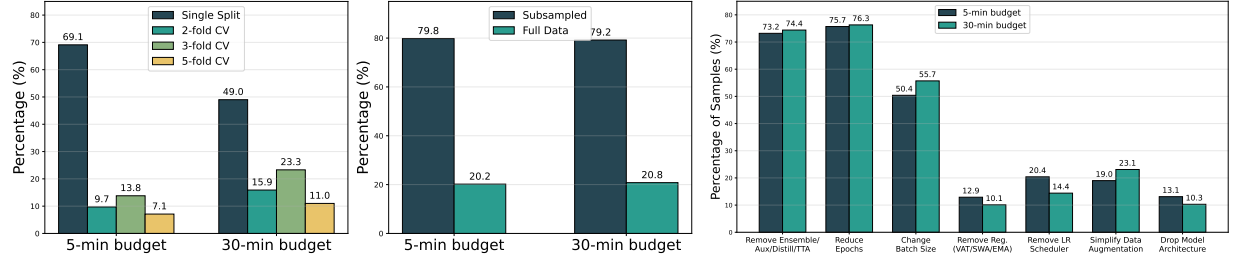


Figure 4: What strategy the Agentic RPM uses. The Agentic RPM frequently runs simplified versions of the original candidates. Agents save time by running reduced cross-validation split counts (left) and sub-sampling the training data (middle). Both agents employ similar training adaptations, such as removing ensembling, reducing epochs and lowering the batch size.

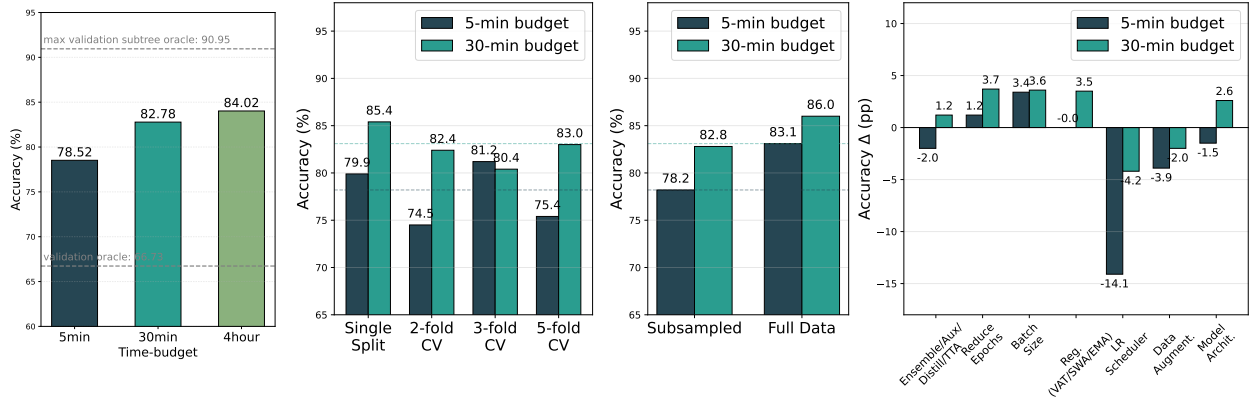


Figure 5: How the Agentic RPM’s strategy affects performance. Increasing the time budget leads to better performance, but with diminishing returns (leftmost). Using more cross-fold validation splits does not correlate with better selection accuracy (middle left). Using full data brings an advantage over using subsampled data (middle right). Most training adaptations improve selection accuracy, except for the removal of the learning rate scheduler and simplification of the data augmentation approach (rightmost).

Proxy Strategy Analysis To understand how agents identify promising candidates under strict time constraints, we use regular expression keyword matching to analyze the agent-generated Python code. Because full execution is impossible within 5- or 30-minute limits, agents actively simplify candidate code across validation, data sampling, and training. As shown in Figure 4, both budgets favor single-split validation, data subsampling, and training adaptations (e.g., removing ensembles, reducing epochs, and lowering batch sizes), though the 30-minute agent utilizes multi-fold cross-validation more frequently. Notably, we provide some hints on possible proxy strategies in the prompt to the pilot experiment, as shown in Appendix Fig. 13 so the strategies are not entirely proposed by the pilot-experiment itself.

Our behavioral analysis in Figure 5 reveals three key insights: (1) multi-fold cross-validation does not consistently improve accuracy due to frequent execution timeouts; (2) evaluating on full datasets provides significantly more reliable quality estimations than using subsampled data; and (3) removing learning rate schedulers or data augmentations triggers severe performance drops, as the underlying AIRA-dojo tasks often rely on these exact training-level optimizations to succeed.

6 Related Work

AI research agents and automated ML research. Our work sits within the literature that formalizes machine learning research as an agentic search problem. Early literature for automated scientific discovery and research assistance uses foundation models to generate ideas, write code, run experiments, analyze results, and draft papers (Lu et al., 2024; Yamada et al., 2025; Schmidgall et al., 2025; Baek et al., 2025; Ren et al., 2025; Zheng et al., 2025). Current work introduces AI research agents as search policies over a tree of candidate

ML solutions, where each node corresponds to an executed or proposed solution and edges correspond to mutations, refinements, or other improvement operators (Toledo et al., 2025; Hambardzumyan et al., 2026). This formulation directly motivates our work: RPMs target the core decision problem induced by such search trees, namely predicting which candidate solution should be executed before committing expensive compute. Other recent works improve the agent scaffold by adding modular search, targeted refinement, ideation agents, multi-agent specialization, or improved code interfaces (Nam et al., 2025; Chen et al., 2026; Zhang et al., 2026; Li et al., 2024; Yang et al., 2024). While these works improve agents, scaffolds, or benchmarks, none of them address the dominant cost in this setting, namely *executing* the candidate solutions that the search proposes.

Model-based methods for expensive search. In reinforcement learning, world models learn environment dynamics and can be used for planning (Schrittwieser et al., 2020) or policy learning from model-generated rollouts (Sutton, 1991; Ha and Schmidhuber, 2018; Hafner et al., 2023). At the *meta*-level, related methods include Bayesian optimization Snoek et al. (2012), early stopping and model-guided search (Li et al., 2018; Falkner et al., 2018), population-based online selection (Jaderberg et al., 2017), learned hyperparameter optimization (Chen et al., 2022), and learned optimization (Andrychowicz et al., 2016). Metz et al. (2022) favor task configurations predicted to expedite optimizer meta-training, Goldie et al. (2025) use a distillation objective as opposed to online evaluation due to the time-cost of training models with every proposed algorithm, and Wolf et al. (2026) present model-based meta-learning for algorithm discovery. These works share our goal of reducing expensive evaluation, yet none use preference ranking based on solution code, reasoning, and partial results to decide which proposed algorithm to evaluate next.

Preference, judge, and reward models. At its core, the RPM is a preference model over candidate solutions. Reward models trained from human preferences are central to reinforcement learning from human feedback (Christiano et al., 2017; Stiennon et al., 2020; Ouyang et al., 2022) and their objectives connect to paired-comparison and ranking formulations (Bradley and Terry, 1952; Liu, 2009). Related preference supervision can also be constructed from expert trajectories without reward-model training or online environment rollouts (Chen and Yuille, 2026). Additionally, strong LLM judges can approximate human preferences on opened responses, subject to known biases (Zheng et al., 2023). Recent work derives continuous scores from scoring-token logits to reduce ties and rank candidates without additional verifier training (Kwok et al., 2026), while Wang et al. (2026) argue that fixed coding-agent rewards can saturate as generators improve. These methods mostly evaluate completed responses, trajectories, or agent outputs, whereas the RPM ranks candidate solutions before expensive full training and evaluation.

Closest to our setting, Zheng et al. (2026) also predict a pairwise preference between unexecuted ML solutions, but condition on a separately prepared, static data report and the two candidate code snippets, rather than on nodes from the surrounding search tree; the RPM instead grounds each comparison in the live search tree and the validation scores of solutions already executed. Goldie et al. (2026) propose training a judge or reward model to select promising leaves in tree-search research agents, but leave it unimplemented. The RPM realizes that proposal without model training, and extends it: where agentic verifiers probe code or user interfaces (Wang et al., 2026), our agentic variant runs small-scale *pilot ML experiments*, partial training and evaluation runs, and chooses based on their measured results.

7 Limitations

- While our evaluations operate under the assumption that LLM inference calls will incur negligible cost in the future, in practice, current LLM inference incurs real-time latency. For the Inference-only RPM, which uses a self-hosted Qwen3.6-27B, inference latency totals 0.660 hours per 24-hour end-to-end run. Adjusting for this time budget yields a normalized score of 0.708 at 23.34 hours, a negligible drop from 0.711. Nevertheless, exact latency and monetary overhead vary by hosting infrastructure and LLM size.
- As the offline data come from prior greedy AIRA-dojo runs (with different LLM backbones and, by design, different task modalities than the online setting), they are off-policy and biased relative to the online target, including the subtree-max label bias noted in Section 4, and thus our main claims rest on the end-to-end results.
- We limit our RPM integration to the *child creation* stage only. We also report initial results integrating RPMs within *final-node selection* in Appendix D. However, we do not observe significant improvement

over validation-based selection, given the Hidden Consistent Evaluation protocol’s strong test-validation generalization (Hambardzumyan et al., 2026). We leave RPM integration within *parent-selection* to future work.

- We describe the RPM as scaffold-agnostic because it inspects no scaffold-internal state, but we demonstrate it only in AIRA-dojo’s child-selection step, with a single backbone (Qwen3.6-27B) on a single benchmark. The agentic variant additionally needs a sandboxed clone of the execution environment in which to run pilot experiments. We see no reason the approach would not transfer, but portability to other scaffolds and backbones is part of our future work.

8 Conclusion

An AI research agent can propose a candidate solution far faster than it can evaluate it. We introduced the **AI Research Preference Model (RPM)**, which ranks unexecuted candidates so that the agent can direct its execution budget toward promising candidates. This formulation avoids requiring the model to forecast absolute outcomes: it identifies the most promising candidate without taking on the challenging task of predicting what any candidate would score.

Our results show that RPM-guided candidate selection improves both final performance and research efficiency while keeping the underlying agent backbone and search operators fixed. More broadly, they show that candidate selection is a useful target for test-time compute: agents can invest computation not only in generating candidates, but also in deciding which candidates are worth executing.

AI research often lies on the less favorable side of the asymmetry of verification: candidate solutions are easy to generate but hard to verify through full training and evaluation (Wei, 2025). RPMs are designed to navigate this asymmetry. While they cannot reduce the cost of any single verification, they can reallocate it, spending the same budget on preferred candidates to reach better solutions sooner. By making this allocation part of the research loop, we hope this work encourages research agents that choose which solutions to run as carefully as they design them.

References

- Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G Bellemare. Deep reinforcement learning at the edge of the statistical precipice. *Advances in Neural Information Processing Systems*, 2021.
- Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W. Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando de Freitas. Learning to learn by gradient descent by gradient descent. In *Advances in Neural Information Processing Systems*, volume 29, 2016.
- Anthropic. Claude code overview. <https://code.claude.com/docs/en/overview>, 2025.
- Anthropic. Introducing claude opus 4.8, 2026. <https://www.anthropic.com/news/claude-opus-4-8>.
- Jinheon Baek, Sujay Kumar Jauhar, Silviu Cucerzan, and Sung Ju Hwang. ResearchAgent: Iterative research idea generation over scientific literature with large language models. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6709–6738, Albuquerque, New Mexico, apr 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.naacl-long.342. <https://aclanthology.org/2025.naacl-long.342/>.
- Ralph Allan Bradley and Milton E Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.
- Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Aleksander Madry, and Lilian Weng. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *The Thirteenth International Conference on Learning Representations*, 2025. <https://openreview.net/forum?id=6s5uXNWGIh>.
- Jiefeng Chen, Bhavana Dalvi Mishra, Jaehyun Nam, Rui Meng, Tomas Pfister, and Jinsung Yoon. Mars: Modular agent with reflective search for automated ai research. *arXiv preprint arXiv:2602.02660*, 2026.
- Yixiong Chen and Alan Yuille. Agentic-DPO: From imitation to agentic policy optimization on expert trajectories, 2026. <https://arxiv.org/abs/2607.10601>.
- Yutian Chen, Xingyou Song, Chansoo Lee, Zi Wang, Qiuyi Zhang, David Dohan, Kazuya Kawakami, Greg Kochanski, Arnaud Doucet, Marc’Aurelio Ranzato, Sagi Perel, and Nando de Freitas. Towards learning universal hyperparameter optimizers with transformers. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, 2017.
- Jade Copet, Quentin Carbonneaux, Gal Cohen, Jonas Gehring, Jacob Kahn, Jannik Kossen, Felix Kreuk, Emily McMilin, Michel Meyer, Yuxiang Wei, David Zhang, et al. CWM: An open-weights LLM for research on code generation with world models. *arXiv preprint arXiv:2510.02387*, 2025.
- Stefan Falkner, Aaron Klein, and Frank Hutter. BOHB: Robust and efficient hyperparameter optimization at scale. In *Proceedings of the 35th International Conference on Machine Learning*, pages 1437–1446. PMLR, 2018.
- Alexander D. Goldie, Zilin Wang, Adrian Hayler, Deepak Nathani, Edan Toledo, Ken Thampiratwong, Aleksandra Kalisz, Michael Beukman, Hannah Erlebach, Alistair Letcher, Shashank Reddy, Clarisse Wibault, Theo Wolf, Charles O’Neill, Uljad Berdica, Nicholas Roberts, Saeed Rahmani, Roberta Raileanu, Shimon Whiteson, and Jakob N. Foerster. Discogen: Procedural generation of algorithm discovery tasks in machine learning, 2026. <https://arxiv.org/abs/2603.17863>.
- Alexander David Goldie, Zilin Wang, Jaron Cohen, Jakob Nicolaus Foerster, and Shimon Whiteson. How should we meta-learn reinforcement learning algorithms? In *Reinforcement Learning Conference*, 2025.
- Google DeepMind. Gemini 3.1 pro: A smarter model for your most complex tasks. <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro/>, February 2026. Accessed: 2026-07-19.
- David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

- Karen Hambardzumyan, Nicolas Baldwin, Edan Toledo, Rishi Hazra, Michael Kuchnik, Bassel Al Omari, Thomas Simon Foster, Anton Protopopov, Jean-Christophe Gagnon-Audet, Ishita Mediratta, Kelvin Niu, Michael Shvartsman, Alisia Lupidi, Alexis Audran-Reiss, Parth Pathak, Tatiana Shavrina, Despoina Magka, Hela Momand, Derek Dunfield, Nicola Cancedda, Pontus Stenetorp, Carole-Jean Wu, Jakob Nicolaus Foerster, Yoram Bachrach, and Martin Josifoski. Aira₂: Overcoming bottlenecks in ai research agents, 2026. <https://arxiv.org/abs/2603.26499>.
- Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M. Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, Chrisantha Fernando, and Koray Kavukcuoglu. Population based training of neural networks. *arXiv preprint arXiv:1711.09846*, 2017.
- Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. Aide: Ai-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. <https://openreview.net/forum?id=VTF8yNQM66>.
- Jacky Kwok, Shulu Li, Pranav Atreya, Yuejiang Liu, Yixing Jiang, Chelsea Finn, Marco Pavone, Ion Stoica, and Azalia Mirhoseini. LLM-as-a-Verifier: A general-purpose verification framework, 2026. <https://arxiv.org/abs/2607.05391>.
- Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameeet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018.
- Ziming Li, Qianbo Zang, David Ma, Jiawei Guo, Tuney Zheng, Minghao Liu, Xinyao Niu, Yue Wang, Jian Yang, Jiaheng Liu, Wanjuan Zhong, Wangchunshu Zhou, Wenhao Huang, and Ge Zhang. AutoKaggle: A multi-agent framework for autonomous data science competitions, 2024. <https://arxiv.org/abs/2410.20424>.
- Tie-Yan Liu. Learning to rank for information retrieval. *Foundations and Trends in Information Retrieval*, 3(3): 225–331, 2009.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery, 2024. <https://arxiv.org/abs/2408.06292>.
- Alisia Lupidi, Bhavul Gauri, Thomas Simon Foster, Bassel Al Omari, Despoina Magka, Alberto Pepe, Alexis Audran-Reiss, Muna Aghamelu, Nicolas Baldwin, Lucia Cipolina-Kun, et al. Airs-bench: a suite of tasks for frontier ai research science agents. *arXiv preprint arXiv:2602.06855*, 2026.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. <https://arxiv.org/abs/2601.11868>.
- Luke Metz, James Harrison, C. Daniel Freeman, Amil Merchant, Lucas Beyer, James Bradbury, Naman Agrawal, Ben Poole, Igor Mordatch, Adam Roberts, and Jascha Sohl-Dickstein. Velo: Training versatile learned optimizers by scaling up, 2022. <https://arxiv.org/abs/2211.09760>.
- Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Jinwoo Shin, Sercan O Arik, and Tomas Pfister. MLE-STAR: Machine learning engineering agent via search and targeted refinement. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. <https://openreview.net/forum?id=vS1M06Px6u>.
- Deepak Nathani, Lovish Madaan, Nicholas Roberts, Nikolay Bashlykov, Ajay Menon, Vincent Moens, Amar Budhiraja, Despoina Magka, Vladislav Vorotilov, Gaurav Chaurasia, Dieuwke Hupkes, Ricardo Silveira Cabral, Tatiana Shavrina, Jakob Foerster, Yoram Bachrach, William Yang Wang, and Roberta Raileanu. MLGym: A new framework and benchmark for advancing ai research agents, 2025. <https://arxiv.org/abs/2502.14499>.
- OpenAI. GPT-4o System Card, 2024. <https://cdn.openai.com/gpt-4o-system-card.pdf>. Accessed: 2024-06-07.

- OpenAI. Introducing gpt-oss, 2025. <https://openai.com/index/introducing-gpt-oss/>. Accessed: 2026-08-12.
- Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.525. <https://aclanthology.org/2024.emnlp-main.525/>.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.
- Arkil Patel, Satwik Bhattamishra, and Navin Goyal. Are nlp models really able to solve simple math word problems? In *Proceedings of the 2021 conference of the North American chapter of the association for computational linguistics: human language technologies*, pages 2080–2094, 2021.
- Qwen Team. Qwen2.5: A party of foundation models, September 2024. <https://qwenlm.github.io/blog/qwen2.5/>.
- Qwen Team. Qwen3.6-27B: Flagship-level coding in a 27B dense model, April 2026. <https://qwen.ai/blog?id=qwen3.6-27b>.
- Shuo Ren, Can Xie, Pu Jian, Zhenjiang Ren, Chunlin Leng, and Jiajun Zhang. Towards scientific intelligence: A survey of llm-based scientific agents, 2025. <https://arxiv.org/abs/2503.24047>.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.
- Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using llm agents as research assistants, 2025. <https://arxiv.org/abs/2501.04227>.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.
- Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.
- Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25, 2012.
- Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel M Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. Learning to summarize with human feedback. In *Advances in Neural Information Processing Systems*, 2020.
- Richard S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. In *Proceedings of the Seventh International Conference on Machine Learning*, pages 216–224, 1991.
- Edan Toledo, Karen Hambardzumyan, Martin Josifoski, Rishi Hazra, Nicolas Baldwin, Alexis Audran-Reiss, Michael Kuchnik, Despoina Magka, Minqi Jiang, Alisia Maria Lupidi, Andrei Lupu, Roberta Raileanu, Tatiana Shavrina, Kelvin Niu, Jean-Christophe Gagnon-Audet, Michael Shvartsman, Shagun Sodhani, Alexander H Miller, Abhishek Charnalia, Derek Dunfield, Carole-Jean Wu, Pontus Stenertorp, Nicola Cancedda, Jakob Nicolaus Foerster, and Yoram Bachrach. AI research agents for machine learning: Search, exploration, and generalization in MLE-bench. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. <https://openreview.net/forum?id=RwfrdKSgCE>.
- Binghai Wang, Chenlong Zhang, Dayiheng Liu, Jiajun Zhang, Jiawei Chen, Mingze Li, Mouxiang Chen, Rongyao Fang, Siyuan Zhang, Xuwu Wang, Yuheng Jing, Zeyao Ma, and Zeyu Cui. The verification horizon: No silver bullet for coding agent rewards, 2026. <https://arxiv.org/abs/2606.26300>.
- Jason Wei. Asymmetry of verification and verifier’s rule, 2025. <https://www.jasonwei.net/blog/asymmetry-of-verification-and-verifiers-law>. Accessed: 2026-08-19.
- Theo Wolf, Alexander David Goldie, Jarek Luca Liesen, Uljad Berdica, Mattie Fellows, and Jakob Nicolaus Foerster. Model-based meta-learning for algorithm discovery. In *ICLR 2026 the 2nd Workshop on World Models: Understanding, Modelling and Scaling*, 2026.

- Michael J. Wooldridge and Nicholas R. Jennings. Intelligent Agents: Theory and Practice. *The Knowledge Engineering Review*, 10(2):115–152, 1995.
- Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search, 2025. <https://arxiv.org/abs/2504.08066>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering, 2024. <https://arxiv.org/abs/2405.15793>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Yunxiang Zhang, Kang Zhou, Zhichao Xu, Kiran Ramnath, Yun Zhou, Sangmin Woo, Haibo Ding, and Lin Lee Cheong. Learning to ideate for machine learning engineering agents. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics, Volume 2: Short Papers*, pages 436–447. Association for Computational Linguistics, 2026.
- Jingsheng Zheng, Jintian Zhang, Yujie Luo, Yuren Mao, Yunjun Gao, Lun Du, Huajun Chen, and Ningyu Zhang. Can we predict before executing machine learning agents?, 2026. <https://arxiv.org/abs/2601.05930>.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623, 2023.
- Tianshi Zheng, Zheyang Deng, Hong Ting Tsang, Weiqi Wang, Jiaxin Bai, Zihao Wang, and Yangqiu Song. From automation to autonomy: A survey on large language models in scientific discovery, 2025. <https://arxiv.org/abs/2505.13259>.
- Qihuang Zhong, Kang Wang, Ziyang Xu, Liang Ding, Juhua Liu, and Bo Du. Achieving > 97% on gsm8k: Deeply understanding the problems makes llms better solvers for math word problems. *Frontiers of Computer Science*, 20(1):1–3, 2026.

Appendix

A Inference-Only RPM

A.1 Prompt Optimization

We optimize the Inference-only RPM’s prompt using Automatic Prompt Optimization with no underlying weight updates.

A.1.1 Optimization Configuration

We base our prompt optimization on MIPROv2 from the DSPy framework (Opsahl-Ong et al., 2024). Starting from the handwritten baseline template presented in Fig. 6, the meta-proposer generates 10 candidate prompt variations. The search space is explored via Thompson sampling over per-candidate Beta(1,1) accuracy posteriors across 40 Bayesian minibatch trials. Underperforming templates are progressively pruned, while top-performing survivors undergo deeper validation runs to mitigate optimization-to-holdout shrinkage. We use GPT-5 as both the meta-proposer and the Inference-only RPM. We evaluate against an offline dataset of 990 samples, drawn similarly to the dataset presented in Section 4.2 but without discarding near-ties.

A.1.2 Results

The optimization routine converges on a *Principal Investigator* persona structured around a five-criterion evaluation rubric. This optimized prompt introduces three main shifts from the initial prompt:

- **Structural Evaluation:** Moves from a flat list of decision rules to a structured walkthrough forcing the model to sequentially score problem-model fit and extensibility.
- **Shifting Bug Tolerance:** Replaces strict penalization of code bugs with an evaluation of core ideas, accepting code bugs if the candidate provides a more promising performance ceiling.
- **Strategic Context Utility:** Sharpens context node usage into an explicit mandate to discover open search gaps and actively penalize redundant directions.

The optimized prompt improves selection accuracy from 57.7% to 59.0%.

A.1.3 Prompt Templates

We present the complete prompt templates for the Inference-only RPM before and after the automated prompt optimization procedure in Figures 6 and 7 respectively.

```
You are a strict judge selecting between TWO candidate solutions to the SAME machine learning task.

Your goal is to choose the candidate whose direction of exploration is more likely to eventually lead to a
better long-term best test score, even if further refinements are needed. Focus on long-term potential
rather than immediate performance.

Task description:
```markdown
{task_desc}

Context from various solutions to the same machine learning task. These are NOT the candidates you are
judging.
{context_text}

Candidate A -- Plan:
{plan_A}
Candidate A -- Code:
```

```

{code_A}

Candidate B -- Plan:
{plan_B}
Candidate B -- Code:
{code_B}

Decision rules:

Think about which candidate opens up a more promising search direction for future iterations.

Prefer the candidate that lays better groundwork for eventually achieving the best possible long-term best
test score, not just the one that looks better right now.

Consider whether the approach is extensible, modular, and amenable to iterative improvement.

Prefer correctness and robustness -- a buggy candidate has no long-term potential.

Use context nodes as evidence of what has already been tried and what directions have shown promise.

Do not assume the context nodes are optimal; the new candidates may open better paths.

Output format (STRICT):

Think step by step and provide your reasoning before giving a final answer.

Give a final answer of A for Candidate A and B for Candidate B.

Provide your answer inside a \boxed{{}}, ie \boxed{{A}} or \boxed{{B}}.

```

**Figure 6:** The un-optimized Inference-only RPM prompt prior to the automated prompt optimization procedure

```

You are a principal investigator allocating compute budget to one of two branches. Decide which branch is
more likely to yield the best eventual test score after several iterations. Emphasize extensibility,
fixability, and promise relative to what has already been tried.

Task description:
{task_desc}

Context from other solutions and their scores (not the candidates). Use this to identify promising gaps and
avoid redundant directions:
{context_text}

Candidate A -- Plan:
{plan_A}
Candidate A -- Code:
{code_A}

Candidate B -- Plan:
{plan_B}
Candidate B -- Code:
{code_B}

Step-by-step evaluation:

```

Problem-model fit:

Does each candidate's formulation and objective align with the task? Any risks of leakage or misalignment?  
Note which issues are trivially fixable vs. fundamental.

Extensibility and upgrade path:

How modular is the code? How straightforward is it to add stronger models, features, or training strategies in 1 to 3 iterations?

Learning curve projection:

Based on current choices, estimate how performance might improve over the next few iterations. Identify low-hanging fruit (data cleaning, features, hyperparameters, regularization, architecture changes).

Context-informed novelty:

Relative to {context\_text}, does the candidate explore a fresh, promising region or iterate intelligently on a proven one? Avoid branches that mirror underperforming context without new leverage.

Risk-adjusted potential:

Balance upside (ceiling) against effort/risk to realize it. Bugs are acceptable if the approach is sound and fixes are clear; penalize only for hard-to-remedy conceptual flaws.

Decision policy:

Choose the candidate whose direction offers higher expected long-term best test score and a credible path to get there.

Output format (STRICT):

Provide reasoning following the steps above.

End with a single final answer: A for Candidate A or B for Candidate B.

Provide your answer inside a `\boxed{A}` or `\boxed{B}`.

**Figure 7:** The optimized Inference-only RPM prompt after the automated prompt optimization procedure

## A.2 Model Ensembling

We evaluate three frontier LLMs, GPT-5 (Singh et al., 2025), Claude Opus 4.8 (Anthropic, 2026) and Gemini 3.1 Pro (Google DeepMind, 2026), individually and ensembled, as baselines on the offline ranking evaluation detailed in Section 4.2.

### A.2.1 Single-Model Configuration

We leverage the same prompt produced by our prompt optimization method, and detailed in Appendix A.1. Following the insights from our offline evaluations, detailed in section 5.2.1, we use the maximum reasoning effort for each model and provide the maximum number of context nodes that fit within each model’s context window.

### A.2.2 Ensembling Configuration

For both ensembling configurations, we perform 3 independent rollouts for each baseline model prior to aggregating their predictions.

**Majority Vote** This strategy employs a mechanical aggregation over the baseline models. For each rollout, the final vote of each model’s response is extracted. A simple majority vote determines the final ensemble prediction, with ties broken uniformly at random.

**LLM Arbiter** The arbiter ensemble replaces mechanical vote counting with a high-level consensus call, using Claude Opus 4.8 as the final arbiter. The arbiter receives the original ranking payload (the task description, historical context, and the candidate pair) alongside the anonymized reasoning traces from the three base models. It is instructed to critically evaluate the logical quality of each expert’s argument rather than blindly deferring to the majority choice. The exact template is presented in Fig. 8.

```
{individual_ranking_prompt}

=====
You are the FINAL ARBITER.

Above is the exact task that was given to multiple independent expert judges: a machine-learning task
description, context from prior solutions, and the candidate solutions.

Each expert independently picked the candidate they believe has the higher long-term potential. Their votes
and full reasoning are given below. The experts are anonymized and may disagree with one another;
any individual expert may be wrong. Do NOT defer to a majority -- weigh the quality of the arguments
against the task and the candidates yourself, then commit to the single best candidate.

Expert 1
Vote: {vote_1}
Reasoning:
{reasoning_trace_1}

...

Expert N
Vote: {vote_N}
Reasoning:
{reasoning_trace_N}
=====
Now make the final decision among Candidate A or Candidate B.

Output format (STRICT):
- Briefly explain which expert arguments you found decisive and why.
- Give a single final answer: one letter either A or B.
- Provide your answer inside a \boxed{}, e.g. \boxed{A}.
```

**Figure 8:** LLM Ensembling Arbiter Prompt

### A.2.3 Results

Table 1 summarizes the offline ranking results. Individually, Claude Opus 4.8 and Gemini 3.1 Pro perform nearly identically (around 67.4%), while GPT-5 trails slightly at 64.66%. Ensembling helps smooth out unique model errors. A standard majority vote lifts accuracy to 68.04%, but the LLM-Arbiter achieves the top score of 69.35%, showing that weighing the quality of arguments outperforms a simple vote count.

Model	Accuracy (%)
GPT-5	64.66
Claude Opus 4.8	67.44
Gemini 3.1 Pro	67.40
Ensembling (majority vote)	68.04
Ensembling (arbiter, Claude 4.8)	<b>69.35</b>

**Table 1:** Selection accuracy of the Inference-only RPM across different LLM backbones and ensembling configurations

### A.3 Reasoning Analysis

To better understand the decision-making process of inference-only RPMs, we analyze the generated reasoning traces across pairwise candidate rollouts. Specifically, we categorize the justifications used in the model’s rationale and evaluate how referencing empirical context impacts selection accuracy.

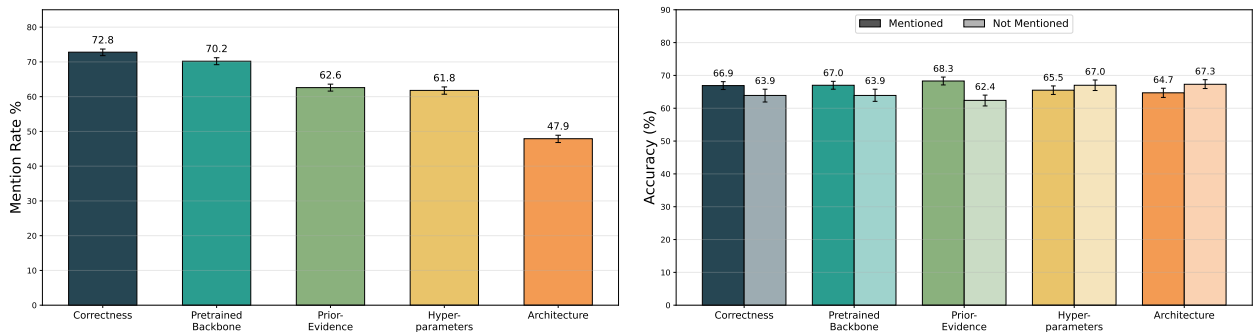
**Evaluation Setup** We evaluate reasoning traces generated from 8,715 pairwise rollouts across three LLM backbones: GPT-5, Claude Opus 4.8 and Gemini 3.1 Pro, all produced as part of the evaluations in [Section A.2.1](#).

**Justification Categorization** Using automated keyword matching, we categorize each reasoning trace into multi-label justification types based on five core categories:

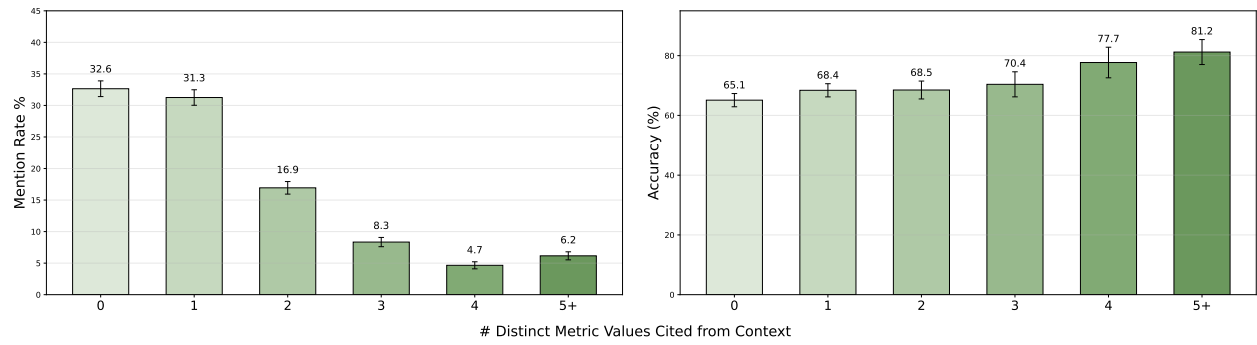
- **Correctness:** Identifying code bugs, syntax errors, or execution flaws.
- **Pretrained Backbone:** Evaluating choices of pretrained model backbones.
- **Prior-Evidence:** Citing measured validation/test metrics from previously explored context nodes.
- **Hyperparameters:** Analyzing learning rates, epoch counts, or optimizer settings.
- **Architecture:** Evaluating structural model modifications (e.g., fusion layers, attention blocks).

As shown in Figure 9, grounding selections in *Correctness* (66.9% vs. 63.9%), *Pretrained Backbone* (67.0% vs. 63.9%), or *Prior-Evidence* (68.3% vs. 62.4%) yields higher selection accuracy compared to when these justifications are omitted. Conversely, relying on abstract *Architecture* justifications decreases selection accuracy from 67.3% to 64.7%.

**Context Citation Analysis** We measure how actively the ranker utilizes historical search trajectories by counting the number of distinct context-node metric values cited in each reasoning trace. Specifically, we parse rollouts using regular expressions to extract, deduplicate, and ground explicit node identifiers and validation metrics cited from the prompt history. As shown in Figure 10, selection accuracy scales monotonically with the



**Figure 9: Reasoning category frequency and selection accuracy impact.** Mention frequency of reasoning categories across reasoning traces (left) and resulting selection accuracy when mentioned versus omitted (right). Citing correctness, pretrained backbones, or prior evidence improves accuracy, whereas citing architecture alone degrades performance.



**Figure 10 : Context citation frequency and selection accuracy impact.** Mention frequency of distinct context metrics across reasoning traces (left) and resulting selection accuracy across citation counts (right). Selection accuracy increases monotonically as more context metrics are cited.

volume of cited context metrics: rollouts citing zero metric values achieve 65.1% accuracy, whereas rollouts citing five or more distinct metric values reach 81.2%.

## B Agentic RPM

### B.1 Algorithm Design and Hyperparameters

In this section we provide more implementation details for agentic RPM. As introduced in [Section 3.2](#), the core of agentic RPM is the pilot experiments. During our study, we find that the agent can be too conservative in scheduling the time budget, leaving a large portion of the time budget unused at the first call of `submit_solution`. Even though we could prompt the agent to run more pilot experiments after the initial submission, the follow-up experiments are often limited to hyperparameter tuning of previous ones. Over conservative time budget scheduling and repetitive follow-up experiments jointly lead to less informative pilot experiments.

To deal with this problem, we use two mechanisms to elicit more informative follow-up experiments. First, we *overstate* the remaining time budget in the prompt (reporting it as several times larger than it truly is), discouraging the agent from stopping prematurely. Second, after each `submit_solution` call, a separate model reviews the findings so far and gives a *feedback* that either proposes the single most informative next experiment or signals that the evidence is already sufficient, in which case we end the loop; its proposal and the remaining budget are then returned to the agent to guide the follow-up experiment. The overall workflow of agentic RPM with the overstating and feedback mechanism is presented at [Algorithm 1](#).

Regarding the hyperparameters, we use the same backbone for the pilot experiment model  $\pi_p$  and the feedback and final prediction model  $\pi_f$  without fine-tuning. The real time budget is  $B = 300s$  even though we overstate that the time budget is  $B' = 2700s$ , which is 9 times larger than the actual value. Every time a pilot experiment is finished and its empirical finding is summarized, we will terminate the loop if 1) the feedback and prediction model  $\pi_f$  does not suggest the next pilot experiment; or 2) the real remaining time budget is less than a threshold  $\theta = 60s$ ; or 3) The maximum number of pilot experiments  $N = 30$  is reached. Notably, the time budget only take effect after all the tool calls in the current turn is executed. Therefore, in practice it is possible that the agentic RPM uses more time than the budget  $B$  if the a tool call is extremely time-consuming.

---

**Algorithm 1** The workflow of the proposed agentic RPM.

---

**Require:** Input query  $q$  containing multiple candidate solutions to rank, pilot-experiment model  $\pi_p$ , feedback and final prediction model  $\pi_f$ , maximum number of pilot experiments  $N$ , time budget  $B$ , stop threshold  $\theta$ .

**Ensure:** Final decision on which candidate to choose  $y_{\text{out}}$ .

```

1: Initialize used time $b = 0$; pilot experiment count $n = 0$; pilot experiment context $x = [q]$; pilot experiment findings $s = []$; last submission flag $F_l = \text{False}$; pending experiment flag $F_p = \text{False}$
2: while True do
3: Generate pilot-experiment model response $y_{\text{asst}} \sim \pi_p(\cdot \mid x)$;
4: Parse possible tool calling $[(\text{tool}_i, \text{para}_i)]_{i=1}^T$ from response y , where T is the number of function call
5: if Find any $\text{tool}_i = \text{python}, i = 1, 2, \dots, T$ then
6: Set pending experiment flag $F_p = \text{False}$
7: if No tools are called and $T = 0$ then
8: Update context $x = x + \text{"Please proceed to the next step using your best judgment....."}$
9: continue
10: Execute the function $y_{\text{tool}}^i = \text{tool}_i(\text{para}_i), i = 1, 2, 3, \dots, T$ and update used time b .
11: Update context $x = x + y_{\text{asst}} + [y_{\text{tool}}^i]_{i=1}^T$
12: if Used time surpasses time budget $b > B$ then
13: Find $i = \text{argmax}_i(\text{tool}_i = \text{python})$ and $j = \text{argmax}_j(\text{tool}_j = \text{submit_solution})$
14: if $i > j$ then
15: Set last submission flag $F_l = \text{True}$
16: Update context $x = x + \text{"TIMEOUT. You have run out of time. Please immediately submit ..."}$
17: continue
18: break
19: if $\text{tool}_T = \text{submit_solution}$ then
20: if Pending experiment flag F_p is True then
21: Update $x = x + \text{"INVALID submit_solution call...You MUST use python tool to run experiment"}$
22: continue
23: $s = s + \text{para}_T$
24: Obtain feedback $f \sim \pi_f(\cdot \mid s)$
25: if The feedback f does not contain next experiment then
26: break
27: if The remaining time budget is less than the stop threshold θ then
28: break
29: Set the pending experiment flag F_p to be True.
30: Update context $x = x + f$
31: if F_l is True then
32: break
33: if The maximum number of pilot experiments is reached $\text{len}(s) \geq N$ then
34: break
35: if The last submission flag F_l is True then
36: break
37: Make the final decision $y_{\text{out}} \sim \pi_f(\cdot \mid s)$
38: Return: Final decision y_{out}

```

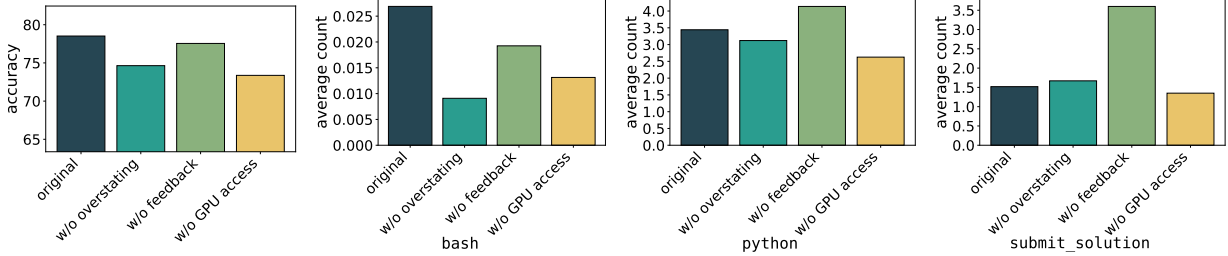
---

## B.2 Design Ablations

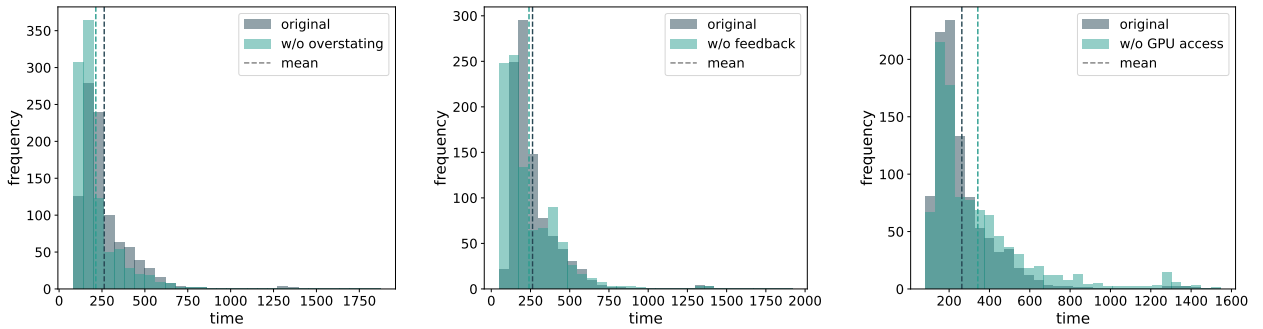
Both end-to-end (Section 5.1) and offline evaluations (Section 5.2.2) demonstrate the efficacy of the Agentic RPM to accelerate and enhance AIRA. To have a better understanding of how different components contribute, in this section we conduct an ablation study on the proposed Agentic RPM. Specifically, we experiment with the following variants:

- **w/o overstating:** The agent is informed with the real remaining time budget without any exaggeration;
- **w/o feedback:** The feedback mechanism is removed and the it receives no planning about the next valuable experiment to run.
- **w/o GPU access:** The agent no longer has access to the H200 GPU and thus can only run pilot experiments with a CPU.

We follow the same offline evaluation setup presented in Section 4.2, and provide the Agentic RPM a 5-minute compute time budget. As presented in Fig. 11, the inclusion of the feedback mechanisms and overstating the provides time budget provides an improvement in selection accuracy. We also observe a significant drop in performance when removing GPU access for the pilot experiments. To further understand the behavior difference between the original agent and its variants, we analyze the average tool call count and the distribution of the pilot-experiments’ running time. The results are presented in Fig. 11 and Fig. 12 respectively. From the figures we can observe that: (1) removing time budget overstating leads to less `python` call and less time-consuming experiments; (2) removing the GPU access increases the time cost of each experiment since the pilot experiments can only be run on CPU in this case; (3) removing feedback mechanism leads to more and briefer experiment, suggesting that the pilot experiments it runs can be trivial and less informative.



**Figure 11:** The performance of three ablation variants at offline evaluation using GPT-5 (leftmost) and their average tool calling count of `bash` (middle left), `python` (middle right), `submit_solution` (rightmost). `bash` is seldom used by any variant. The *w/o feedback* variant calls `python` and `submit_solution` more frequently than other variants.



**Figure 12:** The average execution time distribution of pilot experiments across offline evaluations. Compared with *w/o overstating* variant (left) and *w/o feedback* variant (middle), the original Agentic RPM produces more time-consuming pilot experiments. Without GPU access (right), the average execution time is further increased.

### B.3 Prompt Templates

In this section, we detail the complete set of prompts guiding the agentic RPM workflow. Specifically, we present the prompt templates to initialize the workflow (Figure 13), to provide feedback (Figure 14), and to give a final selection (Figure 15).

#### INTRODUCTION:

You are a Kaggle Grandmaster and Lead Data Scientist acting as a **Strategic Evaluator**. Your task is to analyze two candidate machine learning solutions and **predict which one will achieve a superior test score without actually evaluating on the test set**.

You are given two candidate solutions for a machine learning competition on Kaggle. We do not have access to the test set. Instead, we estimate test performance by running 5-fold cross-validation (CV) on the public training set. However, running the full 5-fold CV on the entire training set is prohibitively time-consuming and beyond our computational budget. Therefore, you must predict which solution would achieve a better test score if it were fully executed, using fast, high-leverage experiments (Pilot Experiments) on the public training set. Remember: your goal is NOT to build the final solution, but to generate code that helps **predict the relative ranking of the two solutions**.

#### TASK DESCRIPTION

...

{task\_desc}

...

You can access the data using standard Python libraries (pandas, datasets, etc.).

The data is already mounted and ready to use — no need to download or prepare it.

**Important:** Pay attention to whether higher or lower values indicate better performance (e.g., accuracy is higher-is-better, and RMSE is lower-is-better).

**Common Pitfalls:** Do NOT use `from transformers import AdamW` (it has been removed). Use `from torch.optim import AdamW` instead.

**Your Resources (as Strategic Evaluator):** {device}

**Time Limit:** {time\_limit}

#### CANDIDATE SOLUTIONS:

##### Option A:

```python

{candidate_a}

...

Option B:

```python

{candidate\_b}

...

.....

#### INSTRUCTIONS FOR PILOT EXPERIMENTS:

You must write a **fast, efficient Python script** (with a target runtime under {time\_limit}). When executed, this script should produce evidence that helps determine which candidate solution would achieve a better test score.

You have full freedom in choosing your analysis strategy. For example, you may:

- Run a simplified or partial cross-validation (e.g., fewer folds, subsampled data).
- Compare key architectural or hyperparameter differences between the two solutions and run targeted ablation experiments.
- Evaluate both solutions on a single train/validation split.
- Perform any other analysis you believe is informative.

#### REQUIREMENT:

Step 1: Use the above information and the provided tools to first understand the task and the candidate solutions, and then write a Python script to generate evidence.

Step 2: Use the provided tools to fix any bugs in your script and run it, adding any additional experiments

needed to strengthen the evidence.

Step 3: You **MUST** summarize your experiment design rationale, experimental results and findings in plain text and submit them with the tool `submit_solution()`.

**Figure 13:** Prompt used for initializing the agentic RPM. The `task_desc`, `device`, `time_limit`, `candidate_a` and `candidate_b` are placeholders.

#### INTRODUCTION:

You are a Kaggle Grandmaster and Lead Data Scientist acting as an **Experiment Planner**. Your task is to decide whether more pilot experimentation is needed, and if so, propose the **single most valuable next experiment** for distinguishing which of the candidate machine learning solutions is most likely to achieve the best test score. You are given a set of candidate solutions for a machine learning competition on Kaggle. You only have access to the public training set but do not have access to the test set. A natural proxy for test performance would be 5-fold cross-validation (CV) score on the public training set. However, running the full 5-fold CV on the entire training set is time-consuming. Therefore, your goal is to design diverse pilot experiments that can approximate relative test performance **with significantly less computation** than standard 5-fold CV.

**Important:** Your job is **only** to decide whether to stop or to design the **next best experiment**. A separate coding agent will execute the experiment and collect results, and a separate judging agent will make the final prediction based on the experimental evidence. You should NOT write any code yourself, and you should NOT make the final prediction yourself.

#### TASK DESCRIPTION

...

{task\_desc}

...

#### ENVIRONMENT & CONSTRAINTS (for the coding agent to follow):

**Data Access:** You can access the data using standard Python libraries (pandas, datasets, etc.). The data is already mounted and ready to use — no need to download or prepare it.

**Metric:** Pay attention to whether higher or lower values indicate better performance (e.g., accuracy is higher-is-better, RMSE is lower-is-better).

**Common Pitfalls:** Do NOT use `from transformers import AdamW` (it has been removed). Use `from torch.optim import AdamW` instead.

**Computation Resources:** {device}

**Time Limit:** {time\_limit}. If you propose a next experiment, it must fit within the remaining time budget with room left for execution reporting and a possible final decision.

#### CANDIDATE SOLUTIONS:

##### Option A:

```
```python
{candidate_a}
```
```

##### Option B:

```
```python
{candidate_b}
```
```

.....

{prev\_findings}

#### INSTRUCTIONS:

You must first decide whether the existing evidence is already sufficient. If yes, recommend stopping. If not, propose exactly **one** next experiment that is expected to provide the highest decision value.

The next experiment should be detailed and specific enough that a coding agent can implement and execute it by strictly following your instructions.

Prefer experiments that reduce the most important unresolved uncertainty about the relative ranking of the

candidate options (Option A, Option B, Option C, ...).

Do NOT propose a trivial repetition of an earlier experiment unless you clearly justify why a stability check is necessary.

Do NOT propose cosmetic variations of previous experiments (for example, the same proxy, same split logic, and same reasoning with only a tiny tweak) unless that variation is specifically needed to resolve an important uncertainty.

Prefer experiments whose outcome could realistically change the current provisional ranking.

If the existing evidence is already strong enough and another experiment is unlikely to add meaningful value, set 'stop: true'.

You have full freedom in designing your strategy. For example, you may:

- Run a simplified or partial cross-validation (e.g., fewer folds, subsampled data).
- Compare key architectural or hyperparameter differences among the candidate solutions and run targeted ablation experiments.
- Evaluate the candidate solutions on a single train/validation split.
- Perform any other analysis you believe is informative.

If you propose a next experiment, it must include:

- **title** (str): A short descriptive name for the experiment.
- **goal** (str): What this experiment is trying to find out.
- **steps** (str): A detailed, step-by-step description of what the coding agent should implement and run.
- **expected\_runtime** (float): A rough estimate of how long this experiment will take in minutes given the computation resources.

**OUTPUT REQUIREMENT:** Output a JSON object with the following schema:

```
```json
{
  "stop": <true_or_false>,
  "reason": "<short explanation>",
  "next_experiment": {
    "title": "<string>",
    "goal": "<string>",
    "steps": "<string>",
    "expected_runtime": <float>
  }
}
```

If "stop": true, then set:

```
```json
"next_experiment": null
```
```

Important:

- Output strictly valid JSON only.
- Do not wrap the JSON in Markdown fences.
- All string values must be valid JSON strings with double quotes.
- "steps" should be a single string. You may use 'n' inside the string for line breaks.

Example Output:

```
{
  "stop": false,
  "reason": "Existing micro-split results are mixed and high-variance. A decisive equal-compute comparison is still needed.",
  "next_experiment": {
    "title": "Matched-step class-balanced holdout",
    "goal": "Run a matched-step comparison between the candidate options under the same compute budget.",
    "steps": "1. Construct a class-balanced holdout split."
  }
}
```

```

        2. Match training steps and optimizer settings.
        3. Run each candidate option under identical budget.
        4. Compare validation accuracy.",
        "expected_runtime": 6.0
    }}
}}
"""

```

Figure 14 : Prompt used to provide feedback in agentic RPM workflow. The `task_desc`, `device`, `time_limit`, `candidate_a`, `candidate_b` and `prev_findings` are placeholders.

INTRODUCTION:

You are a Kaggle Grandmaster and Lead Data Scientist acting as a **Final Prediction Analyst**. Your task is to analyze multiple machine learning solutions and their pilot experiment results to predict which solution will have superior test cross-validation performance.

You have already conducted fast pilot experiments to probe the potential of these multiple solutions. Now you must analyze the empirical evidence from these experiments and make a final prediction.

TASK DESCRIPTION

...

{task_desc}

...

CANDIDATE SOLUTIONS:

Option A:

```
```python
```

{candidate\_a}

...

##### Option B:

```
```python
```

{candidate_b}

...

{execution_output}

INSTRUCTIONS:

Based on the pilot experiment results above, you must predict which candidate solution will achieve the best test cross-validation score when fully implemented.

Consider the following in your analysis:

- **Empirical Evidence:** What do the execution outputs tell you about the performance of each solution?
- **Reliability:** Are the results consistent and reliable, or are there signs of instability?
- **Potential:** Which solution shows more promise for achieving higher validation metrics?
- **Implementation Quality:** Does the output suggest successful execution or potential issues?

OUTPUT REQUIREMENT:

Think step by step and provide your reasoning before giving a final answer. Each candidate solution above is labeled with a letter (Option A, Option B, Option C, ...). Your final answer must be the single letter of the best option, enclosed in `\\boxed{ }` — for example `\\boxed{A}`, `\\boxed{B}`, or `\\boxed{C}`. Output exactly one letter, and only choose from the options listed above.

Example response format:

Analysis: [Your detailed reasoning here]

Final Prediction: `\\boxed{A}`

Figure 15 : Prompt used for the final prediction in agentic RPM workflow. The `task_desc`, `candidate_a`, `candidate_b` and `execution_output` are placeholders.

C End-to-End Evaluations

C.1 Impact of Selection Quality

To determine whether local decision quality directly drives downstream agent performance, we retrospectively analyze the relationship between step-wise candidate selection quality and final end-to-end search scores.

Evaluation Setup Each data point in Figure 16 represents an individual seed run, where the final score is the normalized metric averaged across all AIRS-Bench tasks. To quantify decision quality, we retrospectively evaluate all N candidate solutions generated at each step against ground-truth benchmarks. We define *Selection Advantage* as $S_{\text{chosen}} - \frac{1}{N} \sum_{i=1}^N S_i$, measuring the difference between the chosen candidate’s ground-truth score S_{chosen} and the mean score of all N candidates in the candidate pool. For each seed run, we report the selection advantage averaged across all selection steps and across all tasks.

Findings We find a statistically significant positive correlation between the selection advantage and the final end-to-end performance (Pearson $r = 0.55, p = 0.0007$; Spearman $\rho = 0.56, p = 0.0004$). As expected, random selection yields an average selection advantage near zero. Both preference models consistently improve selection quality over random selection, with the Agentic RPM achieving the highest overall selection advantage and final score. These results confirm that higher per-step candidate selection quality translates directly to better end-to-end AIRA search performance.

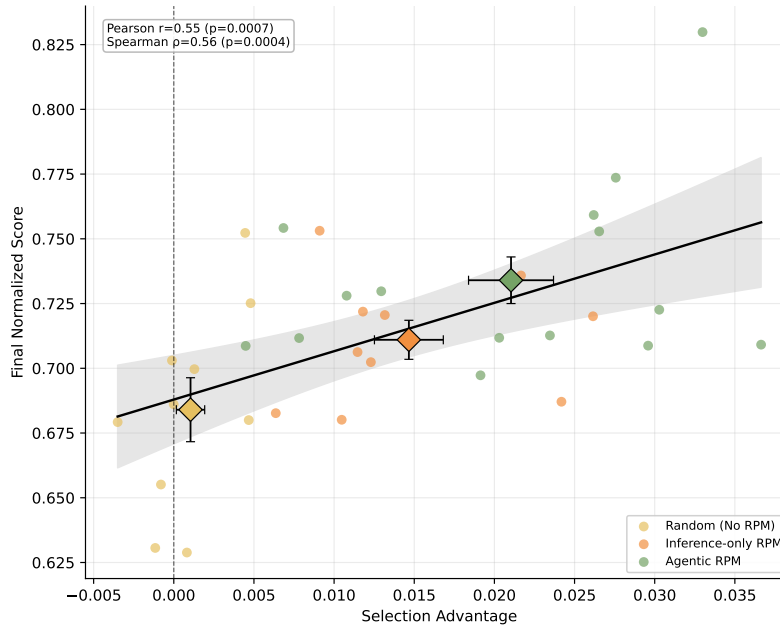


Figure 16: Selection Advantage vs. Final Normalized Score. Final normalized score plotted against step-wise selection advantage across individual seed runs. Large diamonds denote mean values ($\pm 95\%$ CI). Selection advantage strongly correlates with end-to-end performance (Pearson $r = 0.55, p = 0.0007$; Spearman $\rho = 0.56, p = 0.0004$), with Agentic RPM achieving both higher selection advantage and final score than Inference-only RPM and Random selection.

D RPM-Augmented Final Node Selection

As presented in Sections 2.2 and 3.3, an RPM could intervene at three points within an evolutionary search based AIRA’s search: parent selection (PS), child creation (CC) and final-node selection (FNS).

A key distinction separates CC from PS and FNS: CC operates on unexecuted candidates at selection time, while PS and FNS candidates have already been executed and carry validation scores. Thus, a strong selection heuristic (such as greedy selection) can be trivially constructed for PS and FNS. As such, we focus in this manuscript on augmenting CC within an AIRA, but present exploratory initial results on augmenting FNS.

D.1 Evaluation Setup

To evaluate the capability of RPMs to perform FNS, we utilize the search trees produced by the Inference-only RPM End-to-End evaluations across the public text-and-tabular AIRS-BENCH tasks (Sections 4.1 and 5.1). Each datapoint in the evaluation represents a complete AIRA-dojo search tree on a specific task, and the RPM is tasked with selecting the candidate solution with the highest test score from this tree. We use Qwen3.6-27B (Qwen Team, 2026) as the LLM backbone for FNS to maintain model consistency between the original AIRA-dojo operators and selection. We also evaluate GPT-5 as an alternative backbone to validate how stronger selection quality impacts FNS.

To contextualize overall performance, we compare against two global reference baselines: a global *Test Oracle* (the maximum test score in the entire search tree) and a global *Validation Oracle* (the test score of the candidate with the overall highest validation score; the default FNS mechanism of AIRA-dojo). Performance is measured using the average normalized test score of the selected candidate.

D.2 Method

Given an AIRA-dojo search tree, we filter out buggy nodes and retain all valid candidate solutions, with validation and test metrics normalized following Section 2.1. Because an AIRA-dojo search tree can contain hundreds of nodes, we restrict the FNS candidate pool to the N nodes with the highest validation scores, sweeping across pool sizes $N \in \{2, 4, 6, 8, 10\}$. To contextualize performance within each candidate pool, we evaluate against two local baselines: a local *Test Oracle* (N shown) representing the best test score available within the N -candidate pool, and a *Random* (N shown) baseline representing the expected score of a uniform random pick from that pool.

Selection from the candidate pool is conducted via a round-robin tournament, where across 30 repeated matches, candidate subsets of $\min(N, 5)$ are presented to the Inference-only RPM to select between. The candidate accumulating the highest point total across all matches is selected as the tournament winner, with ties broken uniformly at random. The complete prompt template used for the Inference-only RPM during tournament matches is detailed in Fig. 17.

```
You are a strict judge selecting the BEST among 5 candidate solutions to the SAME machine learning task.

Your goal is to choose the candidate most likely to achieve a better test score.

Task description:
```markdown
{{task_description}}
```

Context from various solutions to the same machine learning task. These are NOT the candidates you are
judging.
{{context_nodes_description}}

Candidate A - normalized_validation_metric: {{candidate_a_score}}
Candidate A - Code:
```

```

```python
{{candidate_a_code}}
```

Candidate B - normalized_validation_metric: {{candidate_b_score}}
Candidate B - Code:
```python
{{candidate_b_code}}
```

Candidate C - normalized_validation_metric: {{candidate_c_score}}
Candidate C - Code:
```python
{{candidate_c_code}}
```

Candidate D - normalized_validation_metric: {{candidate_d_score}}
Candidate D - Code:
```python
{{candidate_d_code}}
```

Candidate E - normalized_validation_metric: {{candidate_e_score}}
Candidate E - Code:
```python
{{candidate_e_code}}
```

Decision rules:
- Prefer the candidate most likely to produce a better test score for the given task.
- Prefer correctness, robustness, and task-fit over style or verbosity.
- Use context nodes only as supporting evidence (e.g., what has already been tried, what validation score looked like).
- Do not assume the context nodes are optimal; the new candidates may be better.

Output format (STRICT):
- Think step by step and provide your reasoning before giving a final answer.
- Give a final answer of A, B, C, D or E.
- Provide your answer inside a \boxed{}, ie \boxed{A}, \boxed{B}, \boxed{C}, \boxed{D}, \boxed{E}.

```

Figure 17: The Inference-only RPM’s prompt for final-node selection. For pool sizes N where $N < 5$, the match size is capped at N and the boxed-letter menu shrinks accordingly.

D.3 Results

As shown in Fig. 18, both RPM variants consistently outperform the local random baseline across all pool sizes N . Leveraging GPT-5 yields consistently higher normalized test scores than Qwen3.6-27B, confirming that stronger base reasoning directly improves selection quality. However, the performance of both RPMs degrades as N increases, sliding further below the global *Validation Oracle* (AIRA-dojo’s default selection mechanism) as the judge attempts to separate between an increasingly weaker pool of candidates.

These results illustrate the fundamental challenge of augmenting FNS: once candidates are executed, validation-based selection enables strong heuristics that are difficult to improve upon purely through code inspection. This is further limited given the Hidden Consistent Evaluation protocol’s strong test-validation generalization, the gap between the *Validation Oracle* and the ultimate *Test Oracle* is extremely narrow. While static inference-only judgment struggles to outperform this greedy baseline on executed nodes, future work could deploy more active selection paradigms, such as the Agentic RPM, to run targeted experiments and bridge

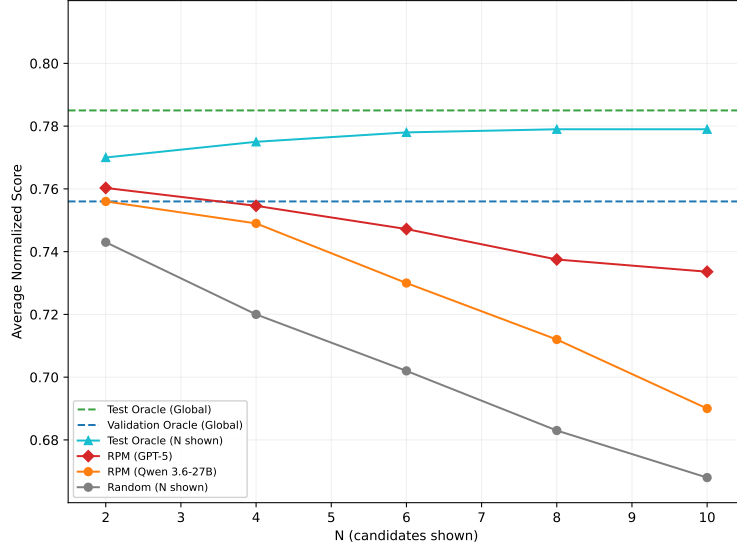


Figure 18 : Final Node Selection performance across candidate pool sizes. Average normalized test score as a function of the top- N validation candidate pool size. Inference-only RPM judges (GPT-5 and Qwen3.6-27B) beat random selection but degrade as N increases, becoming progressively worse than the Validation Oracle (AIRA-dojo’s default selection strategy). The Inference-only RPM with the stronger GPT-5 LLM backbone outperforms that with the Qwen backbone.

the remaining gap to the Test Oracle.